OnDressMe

Flujo de trabajo para cuatro computadoras

Guía de coordinación para backend central, cliente web y dos workers de inteligencia artificial.

Arquitectura distribuida con Tailscale, FastAPI, Redis, React, SAM y Llama

Documento para compartir con los colaboradores del proyecto

1. Objetivo del documento

Este documento define cómo deben colaborar cuatro personas en OnDressMe para evitar incompatibilidades entre código, servicios, modelos y computadoras. Se basa en el documento de arquitectura distribuida del proyecto y en el estado actual del repositorio.

La regla principal es que la Computadora A coordina todo el sistema. La Computadora B solamente consume la API. Las Computadoras C y D procesan trabajos y devuelven sus resultados a A.

2. Arquitectura general

```
Computadora B: cliente/frontend
| HTTP por Tailscale
v
Computadora A: FastAPI + Redis + almacenamiento central
| cola avatar_jobs | cola outfit_jobs
v v
Computadora C: visión/SAM Computadora D: outfits/Llama
```

El PDF de arquitectura propone un patrón Broker/Worker: A recibe las solicitudes, genera identificadores de trabajo, guarda los archivos y distribuye tareas mediante Redis. C realiza visión y D realiza evaluación de estilo y generación de outfits.

Equipo	Responsabilidad	Puerto sugerido
A	FastAPI, Redis, almacenamiento y coordinación	8000 / 6379
B	React, Vite, Three.js y experiencia web	5173
C	Worker de avatar, visión, SAM y colorimetría	8101 opcional
D	Worker de outfits, Ollama y Llama	8102 opcional

3. Estado actual del proyecto

• backend/: API FastAPI, coordinación de trabajos y almacenamiento.

• frontend/: React, Vite, Three.js y React Three Fiber.

• workers/avatar/: worker preparado, pero SAM todavía no está integrado.

• workers/outfits/: worker preparado, pero Ollama/Llama todavía no está integrado.

• shared/contracts/: contrato JSON de trabajos.

• infra/: configuración de Redis mediante Docker Compose.

Actualmente FastAPI crea trabajos y Redis separa las colas avatar_jobs y outfit_jobs. Los workers cambian el estado y reportan que el procesamiento todavía no está implementado. El frontend contiene una demostración visual con botones temporales de estado feliz/triste.

4. Flujo completo de datos

• B envía una imagen o solicitud HTTP a A.

• A guarda los archivos en su almacenamiento persistente y crea un job_id.

• A publica el trabajo en avatar_jobs u outfit_jobs.

• C consume avatar_jobs, ejecuta visión, segmentación, colorimetría o avatar, y devuelve el resultado a A.

• A guarda el resultado visual y, cuando corresponda, genera un trabajo para D.

• D consume outfit_jobs, consulta Llama, valida la respuesta JSON y devuelve la evaluación a A.

• B consulta GET /api/jobs/{job_id}, recibe el resultado final y actualiza la interfaz y el maniquí 3D.

```
Estados permitidos:
queued -> running -> completed
queued -> running -> failed
```

5. Responsabilidades por persona

Persona 1 — Computadora A: backend central y coordinación

Responsabilidad: mantener la API, Redis, almacenamiento, contratos operativos y conexión entre los workers.

Archivos principales:

```
backend/app/api/jobs.py
backend/app/services/job_manager.py
backend/app/schemas/jobs.py
backend/app/core/config.py
backend/app/main.py
backend/app/services/storage/files.py
infra/docker-compose.yml
backend/.env.example
```

• Mantener los estados queued, running, completed y failed.

• Guardar todos los archivos subidos únicamente en A.

• Enviar referencias relativas de archivos; nunca rutas locales C:\... que otros equipos no puedan resolver.

• Implementar la secuencia avatar_jobs -> C -> resultado visual -> outfit_jobs -> D -> resultado final.

• Validar X-Worker-Token y limitar Redis y FastAPI a la red privada de Tailscale.

Persona 2 — Computadora B: cliente/frontend

Responsabilidad: interfaz web, carga de imágenes, consulta de trabajos y visualización del resultado.

Archivos principales:

```
frontend/src/App.jsx
frontend/src/services/api.js
frontend/src/features/mannequin/MannequinScene.jsx
frontend/src/App.css
frontend/src/index.css
frontend/.env.example
```

• Reemplazar los botones temporales por llamadas reales a la API.

• Usar POST /api/avatar/jobs, POST /api/outfits/jobs y GET /api/jobs/{job_id}.

• Mostrar queued, running, completed y failed.

• Interpretar emote_disparador y conectarlo con el maniquí 3D.

• No conectarse directamente a Redis ni ejecutar modelos de IA en B.

```
VITE_API_BASE_URL=http://100.82.214.123:8000
```

Persona 3 — Computadora C: worker de visión/avatar

Responsabilidad: integrar SAM, segmentación de prendas, extracción de colores y análisis corporal/avatar.

Archivos principales:

```
workers/avatar/worker.py
workers/avatar/requirements.txt
workers/avatar/.env.example
```

• Consumir únicamente avatar_jobs.

• Obtener los archivos mediante referencias administradas por A.

• Generar máscara PNG, colores dominantes, tipo de prenda, datos corporales y colorimetría.

• Responder a A con POST /api/internal/jobs/{job_id}/result.

• No conservar el resultado únicamente en C.

Persona 4 — Computadora D: worker de outfits

Responsabilidad: integrar Ollama/Llama, evaluar armonía y generar recomendaciones.

Archivos principales:

```
workers/outfits/worker.py
workers/outfits/requirements.txt
workers/outfits/.env.example
```

• Consumir únicamente outfit_jobs.

• Recibir el perfil, las prendas y los datos visuales procesados.

• Crear un prompt estructurado y consultar Ollama/Llama.

• Validar que la respuesta del modelo sea JSON.

• Enviar el resultado a A y no acceder directamente al frontend.

6. Contrato JSON del resultado

El contrato compartido debe mantenerse estable. El archivo principal es shared/contracts/job.schema.json. Ninguna persona debe cambiar nombres de campos sin avisar y revisar el cambio con el equipo.

```
{
"transaccion_id": "usr_99a8b1_outfit_45",
"perfil_usuario": {
"colorimetria": "Invierno Absoluto",
"morfologia_corporal": "Rectangular"
},
"evaluacion": {
"outfit_valido": false,
"coeficiente_armonia": 0.35,
"emote_disparador": "EMOTE_TRISTE_DESAPROBACION",
"analisis_critico": "La prenda no coincide con la paleta.",
"sugerencia_reemplazo": "Usar azul cobalto."
}
}
```

7. Orden de trabajo recomendado

Fase 0 — Contrato común

Primero trabaja la Persona 1 con revisión de las otras tres. Deben congelarse nombres de colas, estados, job_id, endpoints, errores, resultados y referencias de archivos.

Fase 1 — Preparación de Tailscale

A debe estar disponible por su IP Tailscale. Todos deben comprobar las conexiones B -> A:8000, C -> A:6379, C -> A:8000, D -> A:6379 y D -> A:8000. Redis y FastAPI no deben exponerse a Internet.

Fase 2 — Backend mínimo

La Persona 1 prueba primero /health, creación de trabajos, consulta de estado y actualización interna. Antes de integrar IA debe comprobarse que un trabajo pasa de queued a running y después a completed o failed.

Fase 3 — Workers sin IA

C y D prueban primero la comunicación Redis -> worker -> A. Después integran SAM y Ollama/Llama. Así se separan problemas de red de problemas de modelos.

Fase 4 — Integración frontend

B se conecta al backend cuando los endpoints y el contrato estén estables. Debe subir, consultar, mostrar progreso y representar el resultado.

Fase 5 — Prueba completa

La prueba final debe recorrer B -> A -> C -> A -> D -> A -> B, incluyendo estados, errores, archivo persistente, resultado JSON y emote del maniquí.

8. Organización de archivos y configuración

No deben mover archivos manualmente entre computadoras. Lo recomendable es usar el mismo repositorio Git con ramas separadas. Cada computadora puede tener una copia local del proyecto, pero los archivos .env deben permanecer locales y nunca subirse.

```
A: backend, infra y shared/contracts
B: frontend y copia de shared/contracts
C: workers/avatar y shared/contracts
D: workers/outfits y shared/contracts
```

Archivos sensibles que no deben compartirse en Git:

```
backend/.env
frontend/.env
workers/avatar/.env
workers/outfits/.env
```

9. Ramas e integración Git

```
main
backend-central
frontend-client
worker-avatar
worker-outfits
contracts
```

Orden recomendado de integración:

```
contracts
-> backend-central
-> worker-avatar + worker-outfits
-> frontend-client
-> main
```

No deben modificarse simultáneamente sin coordinación los archivos críticos backend/app/schemas/jobs.py, backend/app/services/job_manager.py y shared/contracts/job.schema.json.

10. Reglas de comunicación

• Toda modificación al contrato debe comunicarse antes de implementarse.

• Cada persona debe indicar qué archivos modificó y qué variables .env necesita.

• Los workers deben devolver errores explícitos; no deben generar resultados simulados.

• A es la única fuente central de archivos persistentes y estados.

• B no debe conocer Redis, rutas locales ni detalles internos de los modelos.

• Las pruebas deben realizarse primero por servicio y después con el flujo completo.

11. Lista de comprobación final

• Todos los equipos aparecen conectados en Tailscale.

• FastAPI responde desde la IP Tailscale de A.

• Redis acepta conexiones únicamente desde Tailscale.

• B puede crear y consultar trabajos.

• C procesa avatar_jobs y devuelve resultado o error.

• D procesa outfit_jobs y devuelve JSON válido.

• Los archivos se almacenan en A.

• Los estados de trabajo son consistentes.

• El frontend muestra progreso y errores.

• El maniquí interpreta el emote_disparador.

• Ningún .env ni storage/ se sube al repositorio.

12. Conclusión

La base actual de OnDressMe ya está separada para cuatro computadoras, pero se encuentra en fase de infraestructura. A está preparado como coordinador, B tiene una interfaz 3D demostrativa y C/D son workers de prueba. El backend debe comenzar primero; después se validan los workers y finalmente se conecta el frontend al flujo real. La prioridad técnica es completar el contrato JSON y encadenar correctamente el resultado de C hacia D.